

Module 16: Circular Singly Linked List Insertion

1. Current Progress Analysis

You are now learning **Module 16**.

Before this module, you already studied the main ideas needed to understand circular singly linked list insertion.

Previous idea	Why it matters in Module 16
Arrays vs linked lists	You understand that linked lists grow by creating nodes and changing links.
Node structure	You know that each node stores <code>data</code> and <code>next</code> .
Pointer movement	You know that <code>p = p->next</code> moves a pointer to the next node.
Traversal	You know how to visit nodes one by one.
Normal singly insertion	You know the safe insertion pattern: <code>t->next = p->next; p->next = t;</code>
Circular singly lists	You know that the last node points back to <code>Head</code> , not to <code>NULL</code> .
<code>do-while</code> loops	You know why circular lists need a loop that runs at least once.

In a normal singly linked list, the list ends at `NULL`:

```
10 -> 20 -> 30 -> NULL
```

In a circular singly linked list, the last node points back to the first node:

```
Head
|
v
10 -> 20 -> 30
^         |
|         +----->
```

The main rule is:

```
last->next == Head
```

Module 16 teaches how to insert a new node into this circular structure without breaking the circle.

2. Big Idea of Module 16

Module 16 is about **inserting a node into a circular singly linked list**.

Insertion means:

Create a new node and connect it into the list by changing `next` links.

In a circular singly linked list, insertion is slightly different from normal singly linked list insertion because there is no `NULL` at the end.

A circular singly linked list must always keep this structure:

```
first node -> second node -> ... -> last node -> first node again
```

So when we insert a node, we must make sure the list still comes back to `Head`.

3. Circular Singly Linked List Reminder

A circular singly linked list uses the same node structure as a normal singly linked list.

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Each node has two parts:

Part	Meaning
<code>data</code>	The value stored in the node.
<code>next</code>	The address of the next node.

The difference is not the node structure.

The difference is the final link.

Normal singly linked list

```
10 -> 20 -> 30 -> NULL
```

The last node points to `NULL`.

Circular singly linked list

```
10 -> 20 -> 30
^          |
|          |
|          |
```

The last node points back to the first node.

4. Meaning of `Head`

In circular linked lists, the starting pointer is often called `Head`.

```
struct Node *Head = NULL;
```

`Head` points to the first node of the circular list.

Example:

```
Head
|
v
8 -> 3 -> 9 -> 6 -> 2
^          |
|          |
```

The list starts at `Head`, but it does not naturally end at `NULL`.

So we use this rule:

Start at `Head`. Stop when you come back to `Head`.

5. Empty, One-Node, and Multi-Node Circular Lists

Case 1: Empty circular list

An empty circular singly linked list has no nodes.

```
Head == NULL
```

Picture:

```
Head -> NULL
```

There is no node to visit.

Case 2: One-node circular list

A one-node circular list has one node, and that node points to itself.

```
Head
|
v
10
^|
||
|_|
```

In code:

```
Head->next = Head;
```

This means:

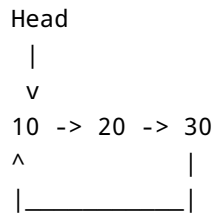
The node after `Head` is `Head` itself.

Important difference:

List type	Condition
Empty circular list	<code>Head == NULL</code>
One-node circular list	<code>Head != NULL</code> and <code>Head->next == Head</code>

Case 3: Multi-node circular list

Example:



Here:

30->next == Head

The last node points back to the first node.

6. Index Convention for Insertion

Module 16 uses this insertion index convention:

Index	Meaning
0	Insert before <code>Head</code> . The new node becomes the new <code>Head</code> .
1	Insert after the first node.
2	Insert after the second node.
3	Insert after the third node.
n	Insert after the last node in a list of length <code>n</code> .

Example list:

8 -> 3 -> 9 -> 6 -> 2 -> back to 8

This list has 5 nodes.

So valid insertion indexes are:

0, 1, 2, 3, 4, 5

Meaning:

Index	Meaning if inserting 10
0	Insert before 8. New list starts with 10.
1	Insert after 8.
2	Insert after 3.
3	Insert after 9.
4	Insert after 6.
5	Insert after 2, the last node.

Invalid indexes are:

-1, 6, 100

If the list has `n` nodes, valid indexes are:

0 to n

7. Why Index Validation Is Needed

Before inserting, we must check whether the index is valid.

The validation rule is:

```
if (index < 0 || index > Length(Head)) {  
    return;  
}
```

This means:

Condition	Meaning
<code>index < 0</code>	Negative indexes are invalid.

Condition	Meaning
<code>index > Length(Head)</code>	The index is beyond the allowed range.

Example:

8 -> 3 -> 9 -> back to 8

Length is 3.

Valid indexes:

0, 1, 2, 3

Invalid indexes:

-1, 4, 5, 100

If we do not validate the index, pointer `p` may move incorrectly and the program may crash or damage the list.

8. Circular Length Function

To validate indexes, we need to know how many nodes are in the circular list.

In a normal singly linked list, we often count like this:

```
while (p != NULL)
```

But this is wrong for a circular list.

Why?

Because a circular list does not end at `NULL`.

Example:

```
8 -> 3 -> 9
^       |
|_____|
```

If we use:

```
while (p != NULL)
```

then `p` may keep moving forever.

So circular length must stop when `p` returns to `Head`.

9. Circular Length Code

```
int Length(struct Node *h) {
    if (h == NULL) {
        return 0;
    }

    int len = 0;
    struct Node *p = h;

    do {
        len++;
        p = p->next;
    } while (p != h);

    return len;
}
```

10. Explanation of the Length Function

Function header

```
int Length(struct Node *h)
```

This function receives a pointer to the first node of the circular list.

It returns the number of nodes.

Empty list check

```
if (h == NULL) {  
    return 0;  
}
```

If the list is empty, there are no nodes.

So length is `0`.

Create a counter and pointer

```
int len = 0;  
struct Node *p = h;
```

`len` counts the nodes.

`p` moves through the circular list.

We use `p` instead of moving `Head` directly.

Count nodes with `do-while`

```
do {  
    len++;  
    p = p->next;  
} while (p != h);
```

This means:

1. Count the current node.
2. Move to the next node.
3. Stop when you return to the starting node.

11. Why `do-while` Is Used

At the beginning:

```
p == h
```

If we write:

```
while (p != h)
```

then the loop will not run even once, because `p` is already equal to `h` at the start.

That would incorrectly return `0` for a non-empty list.

So we use `do-while` because it runs the body first and checks the condition after.

The idea is:

Visit the first node first. Then move. Then check if we returned to the first node.

12. Dry Run of Length

List:

```
Head
|
v
8 -> 3 -> 9 -> 6 -> 2
^                               |
|_____|
```

Call:

```
Length(Head)
```

Start:

```
p = Head, so p points to 8
len = 0
```

Dry run:

Step	p points to	Action	len
1	8	Count 8, move to 3	1
2	3	Count 3, move to 9	2
3	9	Count 9, move to 6	3
4	6	Count 6, move to 2	4
5	2	Count 2, move back to 8	5

Now `p == Head`, so the loop stops.

Final result:

Length = 5

13. Display Function for Testing

Before testing insertion, we need a display function.

A normal linked list display uses:

```
while (p != NULL)
```

But that is wrong for a circular list.

Circular display should stop when it returns to `Head`.

14. Circular Display Code

```
void Display(struct Node *h) {
    if (h == NULL) {
        return;
    }

    struct Node *p = h;

    do {
        printf("%d ", p->data);
        p = p->next;
    }
```

```
    } while (p != h);

    printf("\n");
}
```

15. Explanation of Display

Empty list check

```
if (h == NULL) {
    return;
}
```

If the list is empty, there is nothing to print.

Use a temporary pointer

```
struct Node *p = h;
```

`p` moves through the list.

Do not move `Head` directly.

Print using `do-while`

```
do {
    printf("%d ", p->data);
    p = p->next;
} while (p != h);
```

This prints every node exactly once.

It stops when `p` returns to the starting node.

16. Main Insertion Cases

Circular singly linked list insertion has three main cases:

1. Insert into an empty list.
2. Insert at index in a non-empty list.
3. Insert after a position in a non-empty list.

Each case is slightly different.

Case 1: Insert Into an Empty Circular List

17. Empty List Insertion Idea

Before insertion:

```
Head == NULL
```

Call:

```
Insert(0, 10);
```

We create one new node:

```
10
```

Since this is the only node, it must point to itself.

Final result:

```
Head
|
v
10
^|
||
|_|
```

In code:

```
Head = t;  
Head->next = Head;
```

This means:

1. `Head` points to the new node.
2. The new node points back to itself.

18. Empty List Insertion Code Pattern

```
if (Head == NULL) {  
    Head = t;  
    Head->next = Head;  
    return;  
}
```

Why do we return?

Because insertion is complete.

There is no need to continue to the other cases.

19. Dry Run: Insert Into Empty List

Call:

```
Insert(0, 8);
```

Initial state:

```
Head == NULL
```

Create node `t`:

```
t -> [8 | ?]
```

Do:

```
Head = t;
```

Now:

```
Head -> [8 | ?]
```

Do:

```
Head->next = Head;
```

Now:

```
Head
|
v
8
^|
||
|_|
```

Final list:

```
8 -> back to 8
```

Case 2: Insert at Index 0 in a Non-Empty Circular List

20. Head Insertion Idea

Index **0** means:

Insert before the current **Head**.

So the new node becomes the new **Head**.

Original list:

```

Head
|
v
2 -> 3 -> 4
^       |
|_____|

```

Call:

```
Insert(0, 10);
```

Expected result:

```

Head
|
v
10 -> 2 -> 3 -> 4
^       |
|_____|

```

The new node `10` must become the first node.

But there is an extra circular-list problem.

The old last node must also point to the new `Head`.

Before insertion:

```
4->next == old Head
```

After insertion:

```
4->next == new Head
```

So we must find the last node first.

21. Why We Must Find the Last Node

In a circular singly linked list, each node only points forward.

From `Head`, we can move like this:

```
2 -> 3 -> 4 -> back to 2
```

But we cannot move backward from `Head` to the last node.

So if we want to update the old last node, we must walk through the list until we find it.

The last node is the node whose `next` points to `Head`.

Condition:

```
p->next == Head
```

So to find the last node:

```
struct Node *p = Head;

while (p->next != Head) {
    p = p->next;
}
```

When the loop stops, `p` points to the last node.

22. Head Insertion Link Changes

After finding the last node, use this pattern:

```
t->next = Head;
p->next = t;
Head = t;
```

Meaning:

Step 1

```
t->next = Head;
```

The new node points to the old first node.

Step 2

```
p->next = t;
```

The old last node points to the new node.

Step 3

```
Head = t;
```

`Head` moves to the new node.

Now the new node is the first node of the circular list.

23. Why the Order Matters in Head Insertion

Correct order:

```
t->next = Head;  
p->next = t;  
Head = t;
```

This is safe because:

1. The new node first points to the old list.
2. The old last node then points to the new node.
3. `Head` is updated only after the connections are safe.

Wrong idea:

```
Head = t;  
t->next = Head;
```

This is dangerous because `Head` is changed too early.

Then `t->next = Head` may make the new node point to itself and lose the old list.

Important rule:

Connect the new node to the old list before moving `Head`.

24. Head Insertion Code Pattern

```
if (index == 0) {
    struct Node *p = Head;

    while (p->next != Head) {
        p = p->next;
    }

    t->next = Head;
    p->next = t;
    Head = t;
}
```

25. Dry Run: Head Insertion

Original list:

```
Head
|
v
2 -> 3 -> 4
^         |
|         |
```

Call:

```
Insert(0, 10);
```

Create new node:

```
t -> 10
```

Find the last node:

```
p starts at 2
p moves to 3
```

```
p moves to 4  
stop because 4->next == Head
```

Now:

```
p points to 4  
t points to 10  
Head points to 2
```

Step 1:

```
t->next = Head;
```

Now:

```
10 -> 2
```

Step 2:

```
p->next = t;
```

Now:

```
4 -> 10
```

Step 3:

```
Head = t;
```

Now **Head** points to **10**.

Final list:

```
Head  
|  
v  
10 -> 2 -> 3 -> 4
```

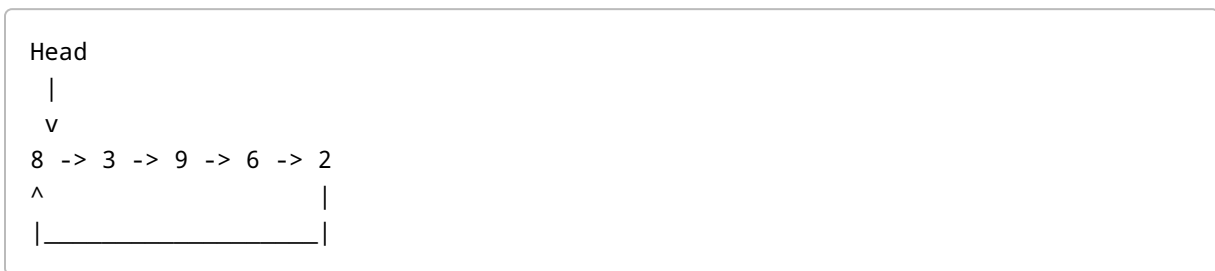


Case 3: Insert After a Position

26. Middle or End Insertion Idea

If `index` is not `0`, we insert after a certain node.

Example list:



Call:

```
Insert(4, 10);
```

Index `4` means:

Insert after the 4th node.

The 4th node is `6`.

Expected result:

```
8 -> 3 -> 9 -> 6 -> 10 -> 2 -> back to 8
```

27. Moving `p` to the Correct Node

For non-head insertion, we move `p` to the node after which the new node should be inserted.

Code:

```

struct Node *p = Head;

for (int i = 0; i < index - 1; i++) {
    p = p->next;
}

```

Why `index - 1`?

Because `p` starts at the first node.

Example:

8 -> 3 -> 9 -> 6 -> 2 -> back to 8

For `index = 4`, we want `p` to reach the 4th node.

Dry movement:

Movement	<code>p</code> points to
Start	8
1	3
2	9
3	6

After `index - 1` movements, `p` points to node `6`.

28. Link Changes for Middle or End Insertion

Once `p` is at the correct node, insert the new node using:

```

t->next = p->next;
p->next = t;

```

This means:

Step 1

```
t->next = p->next;
```

The new node points to the node that originally came after `p`.

Step 2

```
p->next = t;
```

The previous node now points to the new node.

This safely inserts the new node into the chain.

29. Why the Link Order Matters

Correct order:

```
t->next = p->next;  
p->next = t;
```

Wrong order:

```
p->next = t;  
t->next = p->next;
```

Why is the wrong order dangerous?

After this line:

```
p->next = t;
```

`p->next` no longer points to the old next node.

It now points to `t`.

Then this line:

```
t->next = p->next;
```

makes `t->next` point to `t` itself.

That creates a self-loop and may disconnect the rest of the list.

Important rule:

Save the old next link before overwriting it.

30. Middle or End Insertion Code Pattern

```
else {
    struct Node *p = Head;

    for (int i = 0; i < index - 1; i++) {
        p = p->next;
    }

    t->next = p->next;
    p->next = t;
}
```

This works for both middle insertion and end insertion.

31. Dry Run: Middle Insertion

Original list:

```
Head
|
v
8 -> 3 -> 9 -> 6 -> 2
^               |
|_____|
```

Call:

```
Insert(4, 10);
```

Index **4** means insert after the 4th node.

Move **p**:

```
start: p = 8  
move 1: p = 3  
move 2: p = 9  
move 3: p = 6
```

Now **p** points to **6**.

Before insertion:

```
6 -> 2
```

Create new node:

```
10
```

Step 1:

```
t->next = p->next;
```

Since **p->next** points to **2**, now:

```
10 -> 2
```

Step 2:

```
p->next = t;
```

Now:

```
6 -> 10 -> 2
```

Final list:

```
8 -> 3 -> 9 -> 6 -> 10 -> 2 -> back to 8
```

32. Dry Run: End Insertion

Original list:

```
Head
|
v
8 -> 3 -> 9
^       |
|_____|
```

Length is **3**.

Call:

```
Insert(3, 10);
```

Index **3** means insert after the 3rd node.

The 3rd node is **9**, the last node.

Move **p**:

```
start: p = 8
move 1: p = 3
move 2: p = 9
```

Now **p** points to the last node.

Before insertion:

```
9 -> Head
```

Create node **10**.

Step 1:

```
t->next = p->next;
```

Since `p->next` is `Head`, now:

```
10 -> Head
```

Step 2:

```
p->next = t;
```

Now:

```
9 -> 10 -> Head
```

Final list:

```
8 -> 3 -> 9 -> 10 -> back to 8
```

The circle is preserved.

33. Full C Program for Module 16

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *Head = NULL;

int Length(struct Node *h) {
    if (h == NULL) {
        return 0;
    }
}
```

```

    int len = 0;
    struct Node *p = h;

    do {
        len++;
        p = p->next;
    } while (p != h);

    return len;
}

void Display(struct Node *h) {
    if (h == NULL) {
        return;
    }

    struct Node *p = h;

    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != h);

    printf("\n");
}

void Insert(int index, int x) {
    if (index < 0 || index > Length(Head)) {
        return;
    }

    struct Node *t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        return;
    }

    t->data = x;

    if (Head == NULL) {
        Head = t;
        Head->next = Head;
        return;
    }

    if (index == 0) {
        struct Node *p = Head;

```

```

        while (p->next != Head) {
            p = p->next;
        }

        t->next = Head;
        p->next = t;
        Head = t;
    } else {
        struct Node *p = Head;

        for (int i = 0; i < index - 1; i++) {
            p = p->next;
        }

        t->next = p->next;
        p->next = t;
    }
}

int main() {
    Insert(0, 8);
    Insert(1, 3);
    Insert(2, 9);
    Insert(3, 6);
    Insert(4, 2);

    Display(Head);

    Insert(4, 10);
    Display(Head);

    Insert(0, 100);
    Display(Head);

    return 0;
}

```

34. Explanation of the Full Insert Function

Function header

```
void Insert(int index, int x)
```

Meaning:

Part	Meaning
<code>void</code>	The function does not return a value.
<code>Insert</code>	Function name.
<code>index</code>	Position where the new node should be inserted.
<code>x</code>	Value to store in the new node.

Validation

```
if (index < 0 || index > Length(Head)) {  
    return;  
}
```

If the index is invalid, stop immediately.

The list is not changed.

Create the new node

```
struct Node *t = (struct Node *)malloc(sizeof(struct Node));
```

This creates one new node in heap memory.

`t` points to that new node.

Check if memory allocation failed

```
if (t == NULL) {  
    return;  
}
```

If `malloc` fails, no node was created.

So the function stops.

Store the data

```
t->data = x;
```

This stores the value `x` inside the new node.

Empty list case

```
if (Head == NULL) {  
    Head = t;  
    Head->next = Head;  
    return;  
}
```

If the list is empty, the new node becomes the only node.

It points to itself.

Head insertion case

```
if (index == 0) {  
    struct Node *p = Head;  
  
    while (p->next != Head) {  
        p = p->next;  
    }  
  
    t->next = Head;  
    p->next = t;  
    Head = t;  
}
```

This inserts before the old `Head`.

The new node becomes the new `Head`.

Middle or end insertion case

```
else {  
    struct Node *p = Head;  
  
    for (int i = 0; i < index - 1; i++) {  
        p = p->next;  
    }  
  
    t->next = p->next;  
    p->next = t;  
}
```

This inserts after the node at the given position.

It works for both middle insertion and end insertion.

35. Complete Build Dry Run

Suppose we run:

```
Insert(0, 8);  
Insert(1, 3);  
Insert(2, 9);  
Insert(3, 6);  
Insert(4, 2);
```

Step by step:

Step 1

```
Insert(0, 8);
```

List becomes:

```
8 -> back to 8
```

Step 2

```
Insert(1, 3);
```

Index **1** means insert after **8**.

List becomes:

```
8 -> 3 -> back to 8
```

Step 3

```
Insert(2, 9);
```

Index **2** means insert after **3**.

List becomes:

```
8 -> 3 -> 9 -> back to 8
```

Step 4

```
Insert(3, 6);
```

Index **3** means insert after **9**.

List becomes:

```
8 -> 3 -> 9 -> 6 -> back to 8
```

Step 5

```
Insert(4, 2);
```

Index **4** means insert after **6**.

List becomes:

```
8 -> 3 -> 9 -> 6 -> 2 -> back to 8
```

36. More Dry Runs

Dry Run A: Insert **10** at index **4**

Current list:

```
8 -> 3 -> 9 -> 6 -> 2 -> back to 8
```

Call:

```
Insert(4, 10);
```

Index **4** means insert after the 4th node.

The 4th node is **6**.

Result:

```
8 -> 3 -> 9 -> 6 -> 10 -> 2 -> back to 8
```

Dry Run B: Insert **100** at index **0**

Current list:

```
8 -> 3 -> 9 -> 6 -> 10 -> 2 -> back to 8
```

Call:

```
Insert(0, 100);
```

Index `0` means insert before `Head`.

Result:

```
100 -> 8 -> 3 -> 9 -> 6 -> 10 -> 2 -> back to 100
```

Now `Head` points to `100`.

37. Complexity Analysis

Let `n` be the number of nodes in the circular singly linked list.

Operation	Time Complexity	Why
<code>Length(Head)</code>	<code>O(n)</code>	It must count every node once.
Insert into empty list	<code>O(1)</code>	Only one node is created and linked to itself.
Insert at index <code>0</code> in non-empty list	<code>O(n)</code>	We must find the old last node.
Insert in middle	<code>O(n)</code> worst case	We may need to move through many nodes.
Insert at end	<code>O(n)</code> worst case	We must reach the last node.
Extra space	<code>O(1)</code>	Only a few pointer variables are used, not counting the new node.

Important point:

In a normal singly linked list, head insertion is usually `O(1)`.

In a circular singly linked list without a tail pointer, head insertion is `O(n)` because we must update the old last node.

38. Why Head Insertion Is More Expensive in a Circular Singly Linked List

Normal singly linked list:

```
10 -> 20 -> 30 -> NULL
```

To insert at the head:

```
t->next = first;  
first = t;
```

This is $O(1)$.

Circular singly linked list:

```
10 -> 20 -> 30 -> back to 10
```

To insert at the head, the last node must point to the new head.

So we need:

```
30 -> new node  
new node -> old Head  
Head = new node
```

But to update `30`, we must first find `30`.

That takes $O(n)$ time.

39. Important Pointer Rules

Rule 1: Do not use `NULL` as the stopping point

Wrong for circular lists:

```
while (p != NULL)
```

Correct idea:

```
while (p->next != Head)
```

when finding the last node.

Or:

```
do {  
    ...  
} while (p != Head);
```

when visiting all nodes.

Rule 2: Do not move **Head** too early

Wrong idea:

```
Head = t;  
t->next = Head;
```

This can lose the old list.

Correct idea:

```
t->next = Head;  
p->next = t;  
Head = t;
```

Rule 3: Copy old links before overwriting them

Correct:

```
t->next = p->next;  
p->next = t;
```

Wrong:

```
p->next = t;  
t->next = p->next;
```

The wrong order can create a self-loop.

Rule 4: Empty list must be handled separately

Before using:

```
Head->next
```

make sure:

```
Head != NULL
```

If `Head == NULL`, then `Head->next` is invalid.

40. Common Mistakes

Mistake 1: Using linear-list traversal

Wrong:

```
while (p != NULL) {  
    p = p->next;  
}
```

Why wrong?

A circular list may never reach `NULL`.

This can create an infinite loop.

Mistake 2: Forgetting the one-node case

A one-node circular list is not:

```
10 -> NULL
```

It is:

```
10 -> back to 10
```

Code:

```
Head->next = Head;
```

Mistake 3: Not updating the old last node during head insertion

Wrong result:

```
new Head -> old Head -> ... -> old last -> old Head
```

The new node may not be included in the circle correctly.

Correct head insertion must update the old last node:

```
p->next = t;
```

Mistake 4: Inserting with invalid index

If the list has length **5**, valid indexes are:

```
0 to 5
```

Index **6** is invalid.

Always validate:

```
if (index < 0 || index > Length(Head)) {
    return;
}
```

Mistake 5: Changing `p->next` before saving it

Wrong:

```
p->next = t;
t->next = p->next;
```

This makes `t->next` point to `t`.

Correct:

```
t->next = p->next;
p->next = t;
```

41. Comparison: Linear vs Circular Insertion

Feature	Linear singly linked list	Circular singly linked list
Last node points to	NULL	Head
Stop traversal at	NULL	Return to Head
Head insertion	Usually $O(1)$	$O(n)$ without a tail pointer
End insertion	Need to reach last node	Need to reach last node
Empty one-node setup	<code>newNode->next = NULL</code>	<code>newNode->next = newNode</code>
Risk	Losing rest of list	Infinite loop or broken circle

42. Optional Optimization Idea: Tail Pointer

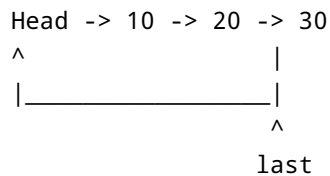
Module 16 usually uses only `Head`.

With only `Head`, inserting at the beginning requires finding the last node.

That takes $O(n)$.

If we also kept a pointer to the last node, often called `tail` or `last`, then head insertion could be faster.

Example:



With `last`, head insertion could use:

```
t->next = Head;
last->next = t;
Head = t;
```

This would be $O(1)$ because we already know the last node.

However, the standard Module 16 version assumes we only have `Head`, so we find the last node by traversal.

43. Simple Mental Model

When inserting into a circular singly linked list, always ask three questions:

1. Is the list empty?
2. Am I inserting before `Head`?
3. Am I inserting after some existing node?

Then use the correct pattern.

Empty list

```
Head = t;
Head->next = Head;
```

Insert before Head

```
find last node p;  
t->next = Head;  
p->next = t;  
Head = t;
```

Insert after position

```
move p to the required node;  
t->next = p->next;  
p->next = t;
```

44. Final Summary

Module 16 teaches how to insert into a circular singly linked list.

The key ideas are:

- A circular singly linked list does not end at `NULL`.
- The last node points back to `Head`.
- Valid insertion indexes are from `0` to `Length(Head)`.
- `Length` must use circular traversal, not `p != NULL`.
- Inserting into an empty list creates a one-node circle.
- Inserting at index `0` makes the new node the new `Head`.
- For head insertion, the old last node must point to the new `Head`.
- Middle and end insertion use the normal safe link pattern.
- The most important link pattern is:

```
t->next = p->next;  
p->next = t;
```

For head insertion, the most important pattern is:

```
t->next = Head;  
last->next = t;  
Head = t;
```

The most important rule is:

After insertion, the last node must still point back to Head.

45. Mastery Checklist

You understand Module 16 if you can do these things:

- Explain why circular lists do not use `NULL` as the stopping condition.
- Write the `Length` function for a circular singly linked list.
- Explain why `do-while` is needed.
- Insert into an empty circular list.
- Insert at index `0` and update `Head` correctly.
- Find the last node using `p->next != Head`.
- Insert in the middle using `t->next = p->next; p->next = t;`.
- Insert at the end while keeping the list circular.
- State valid insertion indexes for a list of length `n`.
- Explain the time complexity of each insertion case.
- Identify common pointer mistakes in circular insertion.

46. Practice Questions

Question 1

A circular list has 4 nodes:

5 -> 10 -> 15 -> 20 -> back to 5

What are the valid insertion indexes?

Answer:

0, 1, 2, 3, 4

Question 2

What does index `0` mean?

Answer:

```
Insert before Head. The new node becomes the new Head.
```

Question 3

Why is this wrong for circular list traversal?

```
while (p != NULL)
```

Answer:

```
Because a circular list may never reach NULL. The pointer may keep moving around the circle forever.
```

Question 4

What condition finds the last node in a circular singly linked list?

Answer:

```
p->next == Head
```

So the loop is:

```
while (p->next != Head) {  
    p = p->next;  
}
```

Question 5

What is the safe link pattern for inserting after node `p`?

Answer:

```
t->next = p->next;  
p->next = t;
```

Question 6

Why is head insertion `O(n)` in this version?

Answer:

```
Because we only have Head. To insert before Head, we must first find the old  
last node so it can point to the new Head.
```

47. One-Line Memory Rule

```
Circular insertion = normal insertion + preserve the link back to Head.
```

That is the core idea of Module 16.